PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

Deep Learning para Finanzas (1FIN58)

Guía de repaso

Métricas de clasificación y estrategias de variación de hiperparámetros

Documento auxiliar de la Tarea Académica 1
Predicción de impago en préstamos de Lending Club

Sebastián Estefano Hernández Muñaqui

Lima, octubre de 2026

Índice

Cómo usar esta guía **3**

1. Preparación de los datos **3**

1.1. Entrenamiento, validación y prueba . . . 3

1.2. Fuga de información . . . 3

1.3. Imputación y codificación *one-hot* . . . 4

1.4. Escalado MinMax . . . 4

2. Métricas de evaluación **5**

2.1. Matriz de confusión . . . 5

2.2. Exactitud, precisión, *recall* y F1 . . . 5

2.3. Pérdida logarítmica (*log-loss*) . . . 6

2.4. Curva ROC, AUC y Gini . . . 6

2.5. Curva precisión–*recall* y precisión promedio (AP) . . . 6

2.6. R^2 sobre probabilidades . . . 7

2.7. ¿Qué métrica depende de qué? . . . 8

3. El umbral de decisión **8**

4. Desbalance de clases y `class_weight` **9**

4.1. Pesos y umbral son casi equivalentes . . . 9

5. Estrategias de búsqueda de hiperparámetros **9**

5.1. Búsqueda en malla (*grid search*) . . . 9

5.2. Búsqueda aleatoria (*random search*) . . . 10

5.3. Búsqueda secuencial (descenso por coordenadas) . . . 10

5.4. Búsqueda iterativa desde una referencia . . . 10

5.5. Criterio de selección y semillas . . . 10

6. Hiperparámetros de optimización **11**

6.1. Descenso por gradiente y sus variantes . . . 11

6.2. Optimizadores . . . 11

6.2.1. SGD 11
6.2.2. SGD con momento 11
6.2.3. RMSprop 11
6.2.4. Adam 12
6.3. Tasa de aprendizaje η 12
6.4. Tamaño de lote B 13
6.5. Épocas 13

7. Hiperparámetros de arquitectura **14**
7.1. La neurona y la capa densa 14
7.2. Funciones de activación 14
7.3. Profundidad y desvanecimiento del gradiente 15
7.4. Ancho (neuronas por capa) 15
7.5. Distribución de neuronas entre capas 16

8. Regularización y control del sobreajuste **17**
8.1. Diagnóstico con curvas de aprendizaje 17
8.2. *Dropout* 17
8.3. Regularización L1 y L2 18
8.4. *Early stopping* y paciencia 18

9. Modelos base **19**
9.1. Regresión logística 19
9.2. Árbol de decisión 19
9.3. *Random Forest* 19

10. Interpretabilidad **20**
10.1. Intrínseca frente a *post hoc* 20
10.2. SHAP 20

11. Tabla resumen de hiperparámetros **21**

12. Preguntas de repaso **21**

Referencias **22**

Cómo usar esta guía

Esta guía reúne, de forma teórica, todos los conceptos que se usaron en el informe final: la preparación de los datos, las métricas, la política de umbral, el tratamiento del desbalance, las estrategias de búsqueda de hiperparámetros, cada hiperparámetro de la red, los modelos base y las técnicas de interpretabilidad. Cada concepto tiene tres capas:

- **Teoría**: la definición y, cuando aplica, la fórmula.
- **Ejemplo**: un cálculo pequeño, hecho a mano, en un recuadro azul.
- **En nuestro proyecto**: cómo apareció el concepto en la tarea, con los números reales de la corrida, en un recuadro gris.

Los recuadros naranjas («Idea clave») resumen lo que conviene recordar. Al final hay una tabla resumen de hiperparámetros (Sección 11) y preguntas de repaso con sus respuestas (Sección 12).

Notación: $y \in \{0,1\}$ es la etiqueta real (1 = impago), $\hat{p} \in [0,1]$ la probabilidad de impago que da el modelo, t el umbral de decisión y $\hat{y} = \mathbb{1}[\hat{p} \geq t]$ la predicción.

1 Preparación de los datos

1.1 Entrenamiento, validación y prueba

Los datos se dividen en tres conjuntos con funciones distintas:

- **Entrenamiento**: el modelo ajusta sus pesos con estos datos.
- **Validación**: se usa para *tomar decisiones*: elegir hiperparámetros, el umbral y el modelo final, y decidir cuándo detener el entrenamiento.
- **Prueba**: se consulta *una sola vez*, al final, para estimar el desempeño con datos nunca usados en ninguna decisión.

La partición es **estratificada** cuando cada conjunto conserva la proporción de clases de la base completa, algo importante cuando una clase es rara.

En nuestro proyecto

Se separó 20 % para prueba y luego 20 % del resto para validación: $0.8 \times 0.8 = 0.64$, es decir, 64 % de entrenamiento (12 739 préstamos), $0.8 \times 0.2 = 0.16$, es decir, 16 % de validación (3 185) y 20 % de prueba (3 981). Los tres conjuntos tienen 14.8 % de impagos.

1.2 Fuga de información

Hay **fuga de información** (*data leakage*) cuando algún dato de validación o prueba influye en el entrenamiento. Un caso típico: calcular la mediana para imputar, o el mínimo y el máximo para escalar, usando toda

la base. Así, el modelo «ve» indirectamente datos que luego se usan para evaluarlo, y las métricas resultan optimistas.

Idea clave

Todo lo que se *aprende* de los datos (medianas, modas, mínimos, máximos, categorías) se ajusta **solo con entrenamiento** (`fit`) y luego se **aplica** a validación y prueba (`transform`).

1.3 Imputación y codificación *one-hot*

La **imputación** rellena valores faltantes: con la mediana en las variables numéricas (robusta a valores extremos) y con la moda en las categóricas. La **codificación *one-hot*** convierte una variable categórica con k categorías en k columnas binarias.

Ejemplo: one-hot

La variable `home_ownership` con valores {RENT, OWN, MORTGAGE} se convierte en tres columnas. Un préstamo con valor OWN queda como $(\text{RENT}, \text{OWN}, \text{MORTGAGE}) = (0, 1, 0)$.

1.4 Escalado MinMax

El escalado MinMax lleva cada variable numérica al intervalo $[0, 1]$:

$$x' = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}, \tag{1}$$

donde $x_{\text{mín}}$ y $x_{\text{máx}}$ se calculan en entrenamiento. Si un dato de prueba supera esos extremos, queda fuera de $[0, 1]$, y eso es correcto: no se reajusta el escalador.

Ejemplo: MinMax (valores ilustrativos)

Si en entrenamiento la tasa de interés va de 5 % a 25 %, un préstamo al 18.3 % queda en $x' = (18.3 - 5)/(25 - 5) = 0.665$. El ingreso anual, de 4 000 a 6 000 000, se lleva a la misma escala $[0, 1]$. Sin escalar, el ingreso dominaría los productos $w \cdot x$ solo por su magnitud.

En nuestro proyecto

En el avance inicial, el MinMaxScaler se construía antes de que existiera la matriz de variables y su salida nunca llegaba a los modelos: las redes se entrenaban sin escalar. La corrección fue un único `ColumnTransformer` ajustado solo con entrenamiento. Resultaron 49 variables: 12 numéricas y 37 indicadoras.

2 Métricas de evaluación

2.1 Matriz de confusión

Con un umbral t, cada predicción cae en una de cuatro casillas:

	Predicho 0 (pagado)	Predicho 1 (impago)
Real 0 (pagado)	VN (verdadero negativo)	FP (falso positivo)
Real 1 (impago)	FN (falso negativo)	VP (verdadero positivo)

En crédito, un **FN** es aprobar un préstamo que no se pagará (pérdida de capital) y un **FP** es rechazar o encarecer a un buen cliente (pérdida de negocio).

2.2 Exactitud, precisión, *recall* y F1

$$\text{Exactitud} = \frac{VP+VN}{VP+VN+FP+FN} \tag{2}$$

$$\text{Precisión} = \frac{VP}{VP+FP} \qquad \text{(de los que marqué como impago, ¿cuántos lo eran?)} \tag{3}$$

$$\text{Recall} = \frac{VP}{VP+FN} \qquad \text{(de los impagos reales, ¿cuántos detecté?)} \tag{4}$$

$$F_1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \tag{5}$$

El F1 es la media armónica de la precisión y el *recall*: solo es alto si ambas lo son. La **exactitud balanceada** promedia el *recall* de cada clase:

$$\text{Exactitud balanceada} = \tfrac{1}{2}\left(\frac{VP}{VP+FN} + \frac{VN}{VN+FP}\right). \tag{6}$$

Ejemplo: la red mejorada en prueba, umbral de validación 0.178

La matriz es VN = 2 522, FP = 870, FN = 293, VP = 296 (3 981 préstamos, 589 impagos).

- Exactitud $= (296+2522)/3981 = 0.708$.
- Precisión $= 296/(296+870) = 0.254$.
- *Recall* $= 296/(296+293) = 0.503$.
- F1 $= 2(0.254)(0.503)/(0.254+0.503) = 0.337$.
- Exactitud balanceada $= \tfrac{1}{2}(0.503 + 2522/3392) = \tfrac{1}{2}(0.503+0.744) = 0.623$.

Idea clave

Con desbalance, la exactitud engaña. Si el 85.2 % de los préstamos se paga, el modelo que siempre dice «pagado» tiene una exactitud de 85.2 % y un F1 de 0. Por eso se priorizan F1, AP y AUC.

2.3 Pérdida logarítmica (*log-loss*)

Es la función que minimiza la red (entropía cruzada binaria):

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\Big]. \tag{7}$$

Castiga con fuerza las predicciones seguras y equivocadas. A diferencia del F1, no depende del umbral, sino de qué tan bien *calibradas* están las probabilidades.

Ejemplo: log-loss

Un préstamo. Si $\hat{p} = 0.342$ y el préstamo era impago, su pérdida es $-\ln 0.342 = 1.07$; si se pagó, es $-\ln 0.658 = 0.42$.

Predictor constante. Un modelo que asigna a todos la prevalencia $\hat{p} = 0.148$ tiene $\mathscr{L} = -[0.148\ln 0.148 + 0.852\ln 0.852] = 0.283 + 0.137 = 0.419$.

En nuestro proyecto

La red de 100 capas terminó con una pérdida de 0.419, justo la del predictor constante: había colapsado a predecir la prevalencia para todos (AUC 0.57). Las redes útiles bajaron a ≈ 0.39.

2.4 Curva ROC, AUC y Gini

La curva ROC grafica el *recall* (tasa de verdaderos positivos) frente a la tasa de falsos positivos $FP/(FP+VN)$ al recorrer todos los umbrales. El **AUC** es el área bajo esa curva y tiene una interpretación probabilística:

$$\text{AUC} = P\big(\hat{p}(\text{impago}) > \hat{p}(\text{pagado})\big), \tag{8}$$

es decir, la probabilidad de que un impago elegido al azar reciba un puntaje mayor que un pagado elegido al azar. Vale 0.5 para un modelo aleatorio y 1 para uno perfecto. El **índice de Gini**, muy usado en riesgo de crédito, es una reescala:

$$\text{Gini} = 2\,\text{AUC} - 1 \in [-1, 1]. \tag{9}$$

2.5 Curva precisión–*recall* y precisión promedio (AP)

La curva PR grafica la precisión frente al *recall*. La **precisión promedio** la resume:

$$\text{AP} = \sum_{n}(R_n - R_{n-1})\,P_n. \tag{10}$$

Su línea base no es 0.5, sino la **prevalencia** de la clase positiva. Con clases desbalanceadas es más exigente que el AUC, porque mide qué tan «limpios» son los primeros préstamos marcados como riesgosos (Saito & Rehmsmeier, 2015).

Ejemplo: AUC y AP a mano

Cinco préstamos ordenados de mayor a menor puntaje, con etiquetas reales $(1,0,1,0,0)$: hay 2 impagos y 3 pagados.

AUC: hay $2 \times 3 = 6$ pares (impago, pagado). El impago del puesto 1 supera a los 3 pagados; el del puesto 3 supera a 2 (los de los puestos 4 y 5). AUC $= 5/6 = 0.833$.

AP: se recorre la lista y se mira cada vez que aparece un impago. En el puesto 1, $P = 1/1$ y $R = 1/2$; en el puesto 3, $P = 2/3$ y $R = 2/2$. AP $= (0.5-0)\cdot 1 + (1-0.5)\cdot \frac{2}{3} = 0.5 + 0.333 = 0.833$.

Gini $= 2(0.833) - 1 = 0.667$.

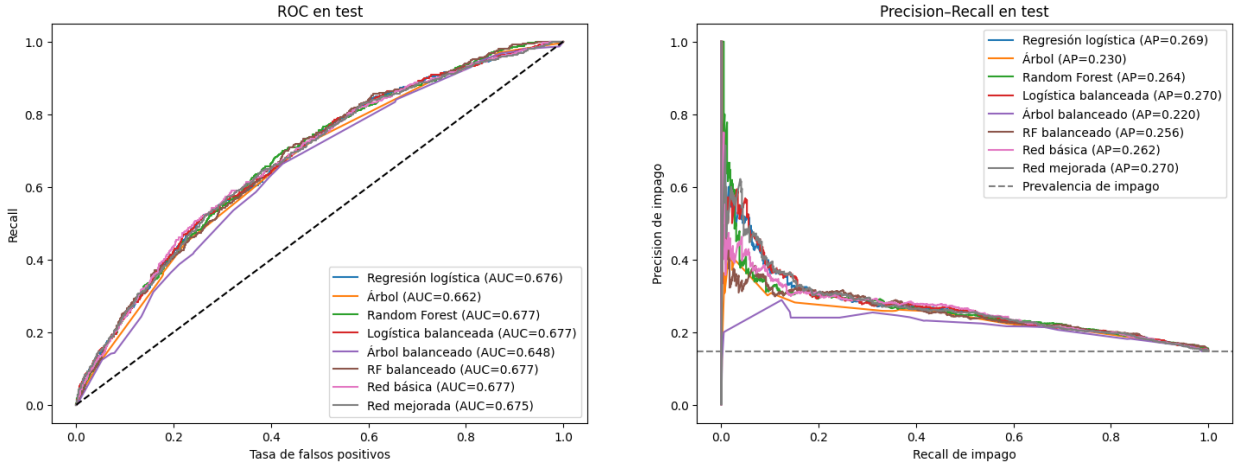


Figura 1. *Curvas ROC (izquierda) y precisión–recall (derecha) de los ocho modelos en el conjunto de prueba del proyecto. La línea discontinua horizontal de la derecha es la prevalencia (0.148), que es el AP de un modelo aleatorio.*

2.6 R^2 sobre probabilidades

Es el coeficiente de determinación calculado entre y y $\hat{p}$: $R^2 = 1 - \sum(y-\hat{p})^2 / \sum(y-\bar{y})^2$. Es una métrica descriptiva, no de clasificación: vale 0 para el predictor constante y puede ser negativa si las probabilidades están desplazadas.

En nuestro proyecto

La red básica con pesos balanceados tuvo $R^2 = -0.70$, aunque su AUC era casi igual al de la versión sin pesos (0.684 frente a 0.686). Los pesos empujan todas las probabilidades hacia arriba: el orden de los préstamos no cambia, pero los valores quedan lejos de 0 y 1.

2.7 ¿Qué métrica depende de qué?

Métrica	¿Depende del umbral?	Línea base aleatoria	Qué mide
Exactitud	sí	prevalencia de la clase mayoritaria (0.852)	aciertos totales
Precisión / *recall* / F1	sí	–	desempeño en la clase impago
Exactitud balanceada	sí	0.5	promedio del *recall* de ambas clases
Log-loss	no	0.419 (constante)	calibración de probabilidades
AUC / Gini	no	0.5 / 0	capacidad de ordenar (ranking)
AP	no	0.148 (prevalencia)	ranking enfocado en la clase rara

Tabla 1. *Resumen de métricas. Las líneas base corresponden a la prevalencia del proyecto.*

3 El umbral de decisión

La red entrega una probabilidad; el umbral t la convierte en decisión. El valor $t = 0.5$ no tiene nada de especial: con una clase rara, las probabilidades de casi todos los préstamos quedan por debajo de 0.5 y el modelo nunca marca impagos.

Bajar el umbral aumenta el *recall* y reduce la precisión. Un criterio común es elegir el t que **maximiza el F1 en validación**, congelarlo y aplicarlo sin cambios a prueba (Figura 2).

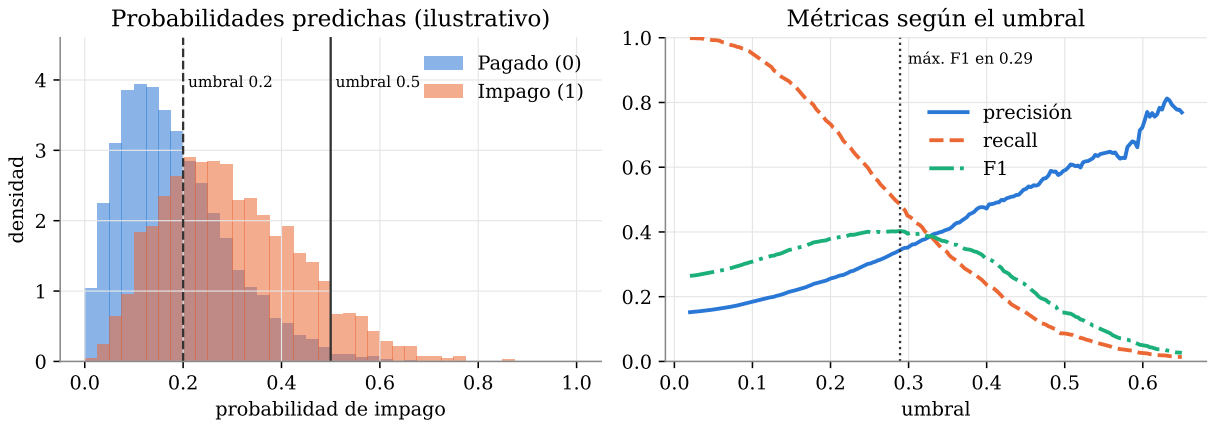


Figura 2. *Ejemplo ilustrativo con datos simulados. Izquierda: las probabilidades de ambas clases se superponen y casi todas quedan bajo 0.5. Derecha: al subir el umbral, el recall cae y la precisión sube; el F1 alcanza su máximo en un umbral bajo.*

Idea clave

Cambiar el umbral **no cambia** el AUC ni el AP, porque no altera el orden de las probabilidades. Solo decide cuántos préstamos se marcan como impago.

En nuestro proyecto

La red mejorada tiene F1 = 0 con $t = 0.5$, porque su probabilidad máxima en toda la prueba es 0.342. Con el umbral de validación $t = 0.178$, su F1 en prueba es 0.337.

4 Desbalance de clases y `class_weight`

Con pesos de clase, la pérdida de cada observación se multiplica por el peso de su clase. Con la opción `'balanced'` de scikit-learn:

$$w_c = \frac{N}{K \cdot N_c}, \tag{11}$$

donde N es el total, $K = 2$ el número de clases y N_c el número de observaciones de la clase c.

Ejemplo: pesos del proyecto

En entrenamiento, $N = 12\,739$, con $N_1 = 1\,883$ impagos y $N_0 = 10\,856$ pagados. $w_1 = 12\,739/(2 \cdot 1\,883) = 3.383$ y $w_0 = 12\,739/(2 \cdot 10\,856) = 0.587$. Equivocarse con un impago «cuesta» $3.383/0.587 \approx 5.8$ veces más.

4.1 Pesos y umbral son casi equivalentes

Ponderar las clases desplaza las probabilidades. En la escala de *log-odds*, el desplazamiento es aproximadamente $\ln(w_1/w_0)$, lo mismo que mover el umbral.

Ejemplo: de un umbral a otro

La logística sin pesos tiene umbral óptimo 0.185, es decir, $\text{logit}(0.185) = \ln(0.185/0.815) = -1.483$. Sumando $\ln(3.383/0.587) = 1.752$ se obtiene 0.269, que equivale a una probabilidad de $1/(1 + e^{-0.269}) = 0.567$. El umbral óptimo de la logística *balanceada* fue 0.564: prácticamente el mismo clasificador.

En nuestro proyecto

Con pesos, el F1 con umbral 0.5 de la red básica subió de 0.004 a 0.355, pero el AP casi no cambió (0.2640 frente a 0.2638). Como la selección se hacía por AP, se conservó la versión sin pesos, y el desbalance se resolvió con el umbral.

5 Estrategias de búsqueda de hiperparámetros

Los **parámetros** (pesos y sesgos) se aprenden con el gradiente. Los **hiperparámetros** (capas, neuronas, tasa de aprendizaje, etc.) se fijan antes de entrenar y se eligen comparando el desempeño en validación.

5.1 Búsqueda en malla (*grid search*)

Prueba todas las combinaciones de una lista de valores. Es exhaustiva, pero el número de entrenamientos crece de forma multiplicativa.

Ejemplo: costo de una malla completa

Con los valores del proyecto (2 de balanceo, 4 optimizadores, 3 tasas, 3 lotes, 4 activaciones, 5 profundidades, 5 anchos, 4 formas, 2 de *dropout*, 3 de regularización, 5 de parada y 3 de épocas) habría $2 \cdot 4 \cdot 3 \cdot 3 \cdot 4 \cdot 5 \cdot 5 \cdot 4 \cdot 2 \cdot 3 \cdot 5 \cdot 3 = 2\,592\,000$ combinaciones. A 100 s por entrenamiento serían unos 8 años de cómputo.

5.2 Búsqueda aleatoria (*random search*)

Sortea combinaciones al azar dentro de rangos. Con el mismo presupuesto suele rendir mejor que la malla, porque casi siempre solo unos pocos hiperparámetros importan de verdad y el muestreo aleatorio prueba más valores distintos de cada uno (Bergstra & Bengio, 2012).

5.3 Búsqueda secuencial (descenso por coordenadas)

Varía **un hiperparámetro por etapa**, elige el mejor y lo deja fijo para las etapas siguientes. El costo es la *suma* de los valores probados, no el producto.

Ventajas: es barata y fácil de interpretar, porque cada etapa aísla un efecto.

Desventajas:

- Ignora interacciones. La mejor tasa de aprendizaje para una red de 1 capa puede no serlo para una de 20.
- El resultado depende del **orden** de las etapas.
- Si una etapa no incluye el valor vigente como candidato, se ve obligada a cambiar aunque empeore.

5.4 Búsqueda iterativa desde una referencia

Cada hiperparámetro se varía *por separado* partiendo siempre de la misma configuración de referencia, y luego se combinan los ganadores. Mide efectos individuales sin arrastre, pero los óptimos individuales no tienen por qué sumarse: al combinarlos, el resultado puede empeorar.

En nuestro proyecto

Se usó la búsqueda secuencial con 43 entrenamientos (unos 73 minutos). El orden fue: balanceo → optimizador → tasa de aprendizaje → lote → activación → capas → neuronas → distribución → *dropout* → regularización → *early stopping* → épocas. Primero se fija la optimización sobre una red pequeña y rápida, luego la arquitectura y al final el control del sobreajuste. En la etapa de *dropout* no se incluyó el valor 0, y el AP bajó de 0.2721 a 0.2520.

5.5 Criterio de selección y semillas

En cada etapa se elige por una métrica de validación. En el proyecto fue el AP, con el F1 de umbral optimizado y el AUC como desempates. Dos cuidados:

- **Una sola semilla** introduce ruido: diferencias de AP de 0.001–0.005 pueden deberse solo a la inicialización aleatoria. Repetir con varias semillas o con validación cruzada da más certeza.

- **Reutilizar validación** en muchas decisiones la vuelve optimista. Por eso la prueba se reserva para el final.

6 Hiperparámetros de optimización

6.1 Descenso por gradiente y sus variantes

El entrenamiento actualiza los pesos en dirección contraria al gradiente de la pérdida:

$$w \leftarrow w - \eta \nabla_w \mathscr{L}. \tag{12}$$

Según cuántos datos se usen para estimar el gradiente, se habla de descenso por lotes completos (toda la base), estocástico (un dato) o por **mini-lotes** (B datos), que es lo habitual.

6.2 Optimizadores

SGD

Aplica directamente $w \leftarrow w - \eta g$, donde g es el gradiente del mini-lote. Es simple, pero lento en valles alargados y sensible a la tasa η.

SGD con momento

Acumula una «velocidad» que promedia los gradientes recientes:

$$v \leftarrow \beta v + g, \qquad w \leftarrow w - \eta v, \qquad \beta = 0.9. \tag{13}$$

Si el gradiente es constante, el paso efectivo tiende a $\eta/(1-\beta) = 10\eta$: acelera en direcciones consistentes y amortigua las oscilaciones.

RMSprop

Divide el paso por la magnitud reciente del gradiente de cada peso:

$$s \leftarrow \rho s + (1-\rho) g^2, \qquad w \leftarrow w - \eta \, \frac{g}{\sqrt{s} + \varepsilon}. \tag{14}$$

Los pesos con gradientes grandes dan pasos más cortos y los de gradientes pequeños, pasos más largos: es una tasa de aprendizaje adaptativa por peso.

Adam

Combina el momento (primer momento m) con RMSprop (segundo momento v) y corrige el sesgo inicial (Kingma & Ba, 2015):

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \tag{15}$$

$$\hat{m} = \frac{m}{1-\beta_1^k}, \qquad \hat{v} = \frac{v}{1-\beta_2^k}, \qquad w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}, \tag{16}$$

con $\beta_1 = 0.9$, $\beta_2 = 0.999$ y k el número de paso.

Ejemplo: primer paso de Adam

Con $g = 0.2$ y $\eta = 0.001$ en el paso $k = 1$: $m = 0.1 \cdot 0.2 = 0.02$ y $v = 0.001 \cdot 0.04 = 0.00004$. Corrigiendo el sesgo, $\hat{m} = 0.02/0.1 = 0.2$ y $\hat{v} = 0.00004/0.001 = 0.04$. El paso es $0.001 \cdot 0.2/\sqrt{0.04} = 0.001$. Si el gradiente fuera $g = 20$, el paso seguiría siendo ≈ 0.001: en Adam, el tamaño del paso lo controla η, no la escala del gradiente.

En nuestro proyecto

Con $\eta = 0.001$ para todos, Adam ganó (AP 0.2661), seguido de Momentum (0.2646) y RMSprop (0.2643). SGD quedó atrás (0.2366) y no se estabilizó en 100 épocas. Esa tasa es pequeña para SGD, que suele necesitar alrededor de 0.01, así que la comparación favorece a los métodos adaptativos.

6.3 Tasa de aprendizaje η

Es el hiperparámetro más influyente en el entrenamiento. Si es muy pequeña, el avance es lento; si es adecuada, la pérdida baja rápido y de forma estable; si es muy grande, los pasos saltan sobre el mínimo, oscilan o divergen (Figura 3).

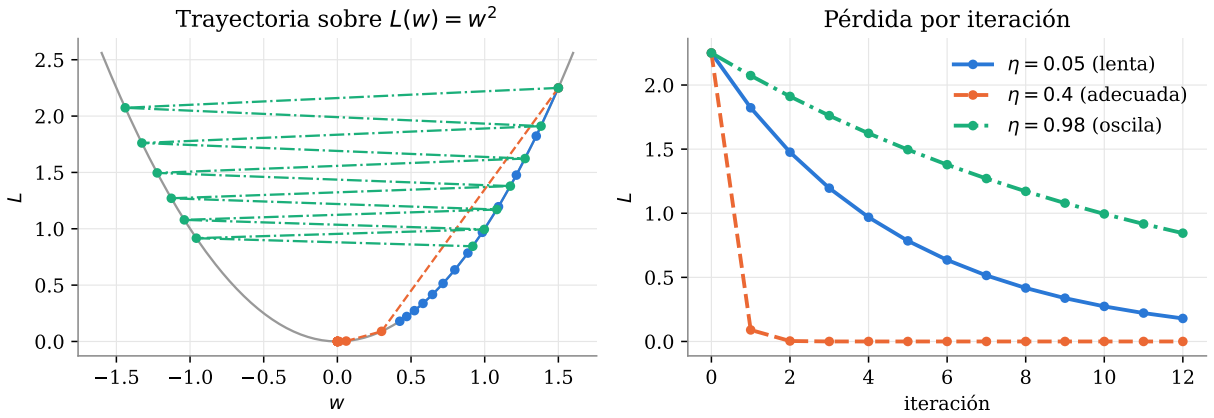


Figura 3. *Descenso por gradiente sobre $L(w) = w^2$ desde $w_0 = 1.5$. La actualización es $w \leftarrow (1-2\eta)w$: con $\eta = 0.05$ avanza poco; con $\eta = 0.4$ converge en pocos pasos; con $\eta = 0.98$ el factor es -0.96 y el peso salta de un lado al otro del mínimo. Con $\eta > 1$ divergería.*

En nuestro proyecto

Con Adam, $\eta = 0.0001$ ganó (AP 0.2690). Con $\eta = 0.01$ el entrenamiento se detuvo en 9 épocas, señal de pasos demasiado grandes.

6.4 Tamaño de lote B

Es el número de observaciones usadas para estimar cada gradiente. Un lote pequeño da gradientes más ruidosos pero más actualizaciones por época, y ese ruido a veces ayuda a generalizar. Un lote grande da gradientes más estables y aprovecha mejor la GPU, pero hace menos actualizaciones.

Ejemplo: actualizaciones por época

Con 12 739 préstamos de entrenamiento: $B = 32$ da $\lceil 12739/32 \rceil = 399$ actualizaciones por época, $B = 64$ da 200 y $B = 128$ da 100.

6.5 Épocas

Una **época** es una pasada completa por el entrenamiento. Pocas épocas producen subajuste; demasiadas, sin control, producen sobreajuste. El número adecuado se decide mejor con *early stopping* (Sección 8.4) que fijándolo a mano.

7 Hiperparámetros de arquitectura

7.1 La neurona y la capa densa

Una capa densa calcula $\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$, donde W es la matriz de pesos, $\mathbf{b}$ el sesgo y g la función de activación. Una capa de n_{in} entradas y n_{out} neuronas tiene $n_{\text{in}}\, n_{\text{out}} + n_{\text{out}}$ parámetros.

Ejemplo: conteo de parámetros

Red básica (49 entradas, 1 capa de 100 neuronas, salida de 1 neurona): $49 \cdot 100 + 100 = 5\,000$ en la capa oculta y $100 + 1 = 101$ en la salida, total 5 101.
Red mejorada (20 capas de 64): la primera tiene $49 \cdot 64 + 64 = 3\,200$; las otras 19 tienen $64 \cdot 64 + 64 = 4\,160$ cada una (79 040 en total); la salida tiene 65. Total: 82 305.

7.2 Funciones de activación

Sin activación no lineal, apilar capas equivale a una sola transformación lineal. Las activaciones comparadas en el proyecto fueron:

$$\text{ReLU: } g(z) = \max(0, z) \qquad \text{tanh: } g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$\text{sigmoide: } g(z) = \frac{1}{1 + e^{-z}} \qquad \text{ELU: } g(z) = \begin{cases} z & z > 0 \\ e^{z} - 1 & z \le 0 \end{cases}$$

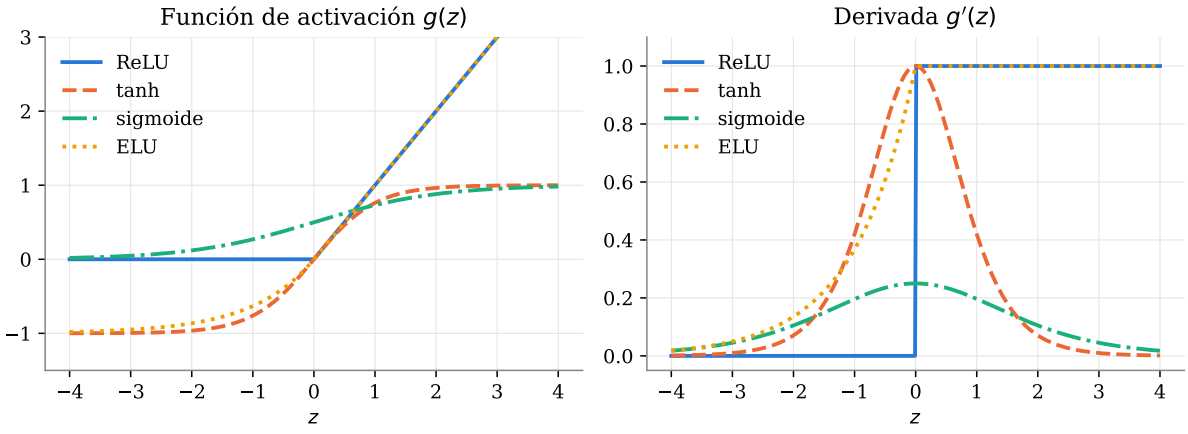


Figura 4. *Activaciones (izquierda) y sus derivadas (derecha). La derivada de la sigmoide nunca supera 0.25 y la de tanh se anula en los extremos (saturación); la de ReLU vale 0 para $z < 0$ («neuronas muertas»), y ELU mantiene un gradiente pequeño en esa zona.*

Activación	Rango	Derivada máx.	Comentario
ReLU	$[0,\infty)$	1	barata, no satura para $z>0$; puede «morir» si $z<0$ siempre
tanh	$(-1,1)$	1	centrada en cero; satura en los extremos
sigmoide	$(0,1)$	0.25	se usa en la *salida* para dar una probabilidad; en capas ocultas favorece el desvanecimiento del gradiente
ELU	$(-1,\infty)$	1	como ReLU, pero suave y con salida negativa; evita neuronas muertas

Tabla 2. *Comparación de las activaciones.*

En nuestro proyecto

Con 1 capa, tanh ganó (AP 0.2765, el mejor de toda la búsqueda), con ELU muy cerca (0.2757). La salida fue siempre una sigmoide, para obtener una probabilidad de impago.

7.3 Profundidad y desvanecimiento del gradiente

Por la regla de la cadena, el gradiente que llega a la primera capa es un producto de derivadas de todas las capas siguientes. Si esas derivadas son menores que 1, el producto se desvanece exponencialmente con la profundidad (Glorot & Bengio, 2010).

Ejemplo: desvanecimiento

Con sigmoide, cada derivada es como máximo 0.25. Tras 20 capas el factor es a lo sumo $0.25^{20} \approx 9\times10^{-13}$: las primeras capas prácticamente no aprenden.

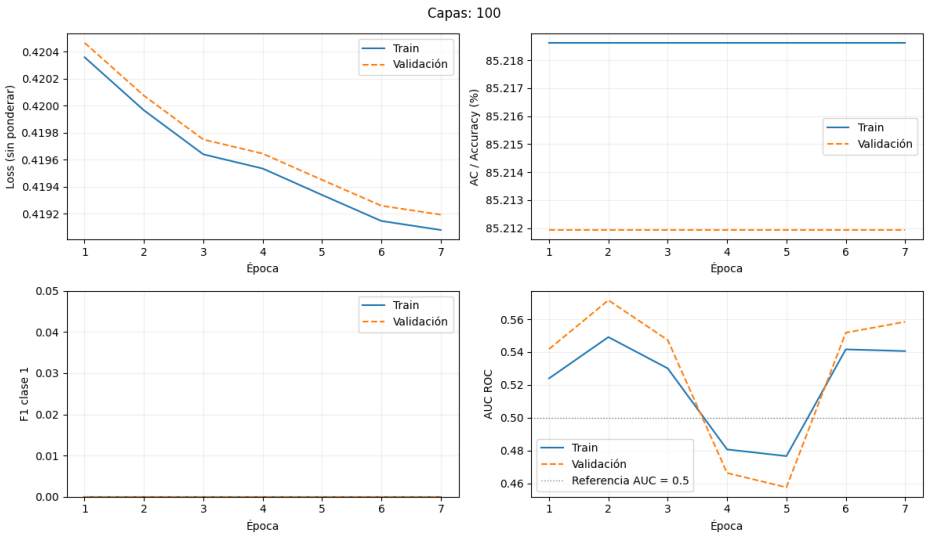


Figura 5. *La red de 100 capas del proyecto: la pérdida casi no cambia (cuarta cifra decimal) y el AUC oscila alrededor de 0.5.*

7.4 Ancho (neuronas por capa)

Más neuronas dan más capacidad de representación, pero también más parámetros, más riesgo de sobreajuste y más costo. En el proyecto, con 20 capas, ganaron 64 neuronas; de 128 a 512 la red dejó de marcar impagos.

7.5 Distribución de neuronas entre capas

Con el mismo número de capas y el mismo total de neuronas, el reparto puede tener distintas formas (Figura 6):

- **Uniforme**: todas las capas iguales.
- **Creciente**: pocas neuronas al inicio y más al final.
- **Embudo (decreciente)**: comprime la información de forma progresiva hacia la salida. Es la forma más usada en clasificación.
- **Cuello de botella**: comprime hasta el centro y luego expande. Es la idea del *autoencoder*.

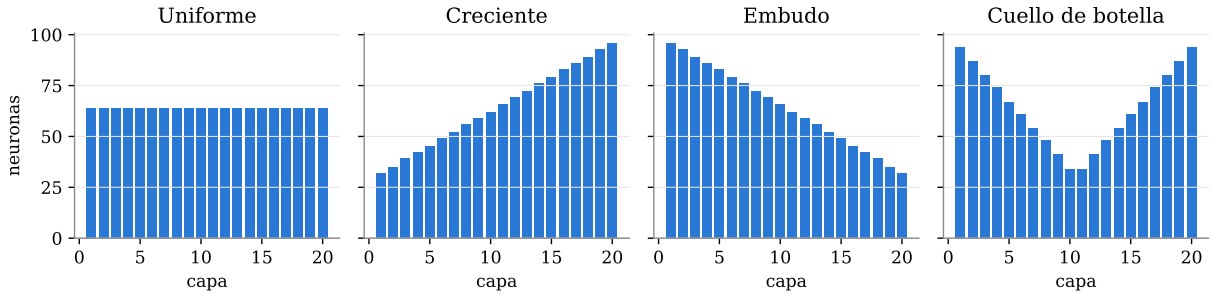


Figura 6. *Las cuatro formas con 20 capas y 1 280 neuronas, calculadas con la función* `distribuir_neuronas` *del notebook. Por ejemplo, el cuello de botella va de 94 neuronas en los extremos a 34 en el centro.*

En nuestro proyecto

Ganó la uniforme (AP 0.2721), seguida del embudo (0.2636), el cuello de botella (0.2619) y la creciente (0.2437).

8 Regularización y control del sobreajuste

8.1 Diagnóstico con curvas de aprendizaje

Comparar las curvas de entrenamiento y validación es la herramienta básica de diagnóstico (Figura 7):

- **Subajuste**: ambas pérdidas altas y planas; el modelo es demasiado simple o no aprende.
- **Buen ajuste**: ambas bajan y se estabilizan juntas.
- **Sobreajuste**: la de entrenamiento sigue bajando y la de validación sube; el modelo memoriza. La **brecha** entre ambas mide cuánto.

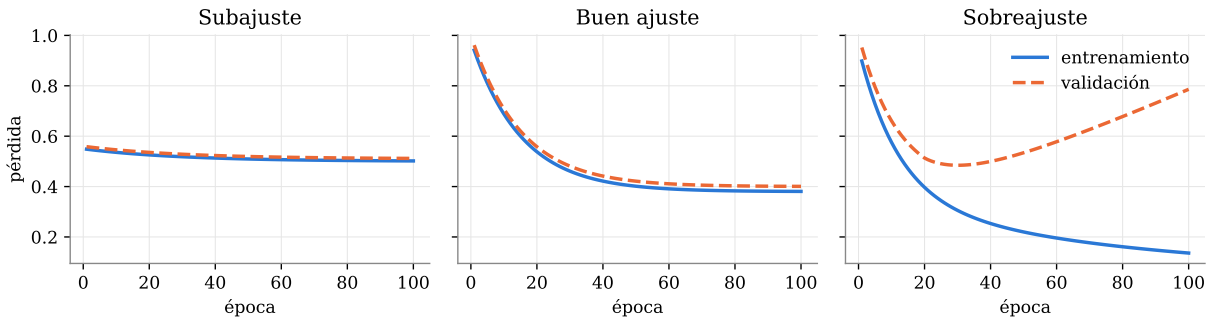


Figura 7. *Curvas esquemáticas de los tres regímenes.*

En nuestro proyecto

En la corrida anterior, la red se entrenó 500 épocas sin control y se sobreajustó: AUC de 0.996 en entrenamiento y 0.582 en prueba. La red final tuvo 0.683 y 0.675.

8.2 *Dropout*

En cada paso de entrenamiento se apaga al azar una fracción p de las neuronas de una capa (Srivastava et al., 2014). Así, la red no puede depender de una neurona concreta y aprende representaciones redundantes; equivale a promediar muchas subredes. En inferencia no se apaga nada. Keras usa *inverted dropout*: durante el entrenamiento escala las neuronas activas por $1/(1-p)$ para que la salida esperada no cambie.

Ejemplo: dropout con $p = 0.25$

Una capa con salidas $(0.8, 0.4, 1.2, 0.6)$ y máscara $(1, 0, 1, 1)$ queda en $(0.8, 0, 1.2, 0.6)/0.75 = (1.067, 0, 1.6, 0.8)$. En promedio se conserva la escala de la salida.

En nuestro proyecto

En la red de 20 capas, *dropout* 0.25 bajó el AP de 0.2721 a 0.2520 y 0.50 hizo colapsar la red (AUC 0.543). Con pocos datos y una red ya limitada, apagar neuronas le quitó capacidad útil.

8.3 Regularización L1 y L2

Se agrega a la pérdida una penalización sobre el tamaño de los pesos (Goodfellow et al., 2016):

$$\mathscr{L}_{\mathrm{L1}} = \mathscr{L} + \lambda \sum_j |w_j|, \qquad \mathscr{L}_{\mathrm{L2}} = \mathscr{L} + \lambda \sum_j w_j^2. \tag{17}$$

La penalización L2 empuja cada peso con una fuerza proporcional a su tamaño ($2\lambda w$), así que achica los pesos sin anularlos. La L1 empuja con fuerza constante (λ signo(w)), así que lleva a cero los pesos pequeños y produce modelos dispersos, una forma implícita de selección de variables.

Ejemplo: L1 frente a L2

Con pesos $w = (0.5, -0.1)$ y $\lambda = 10^{-4}$: la penalización L1 es $10^{-4}(0.5 + 0.1) = 6 \times 10^{-5}$ y la L2 es $10^{-4}(0.25 + 0.01) = 2.6 \times 10^{-5}$. El empuje L2 sobre el peso pequeño es $2\lambda(0.1) = 2 \times 10^{-5}$, y el L1 es 10^{-4}, cinco veces mayor. Por eso L1 anula antes los pesos pequeños.

En nuestro proyecto

L1 con $\lambda = 10^{-4}$ recuperó parte de lo que había perdido el *dropout*: el AP subió de 0.2520 a 0.2640, frente a 0.2586 con L2.

8.4 *Early stopping* y paciencia

Se vigila una métrica de validación y se detiene el entrenamiento cuando no mejora durante **paciencia** épocas seguidas; con `restore_best_weights=True` se recuperan los pesos de la mejor época (Prechelt, 1998). Decide automáticamente el número de épocas y frena el sobreajuste.

Ejemplo: paciencia 5

Si el AP de validación es máximo en la época 18 y no mejora en las épocas 19 a 23, el entrenamiento se detiene en la época 23 y se restauran los pesos de la época 18. Con paciencia 50, se habría seguido hasta la época 68 para terminar restaurando la misma época 18.

En nuestro proyecto

Las paciencias 5, 10, 20 y 50 dieron exactamente el mismo AP (0.2640), porque restauraron los mismos pesos; solo cambió el tiempo, de 89 s a 251 s.

Idea clave

Las técnicas contra el sobreajuste (*dropout*, L1/L2, *early stopping*) solo ayudan si hay sobreajuste. Si el modelo ya generaliza bien, o está limitado por la información de los datos, pueden empeorarlo.

9 Modelos base

9.1 Regresión logística

Modela el *log-odds* de impago como una función lineal de las variables:

$$\ln \frac{p}{1-p} = \beta_0 + \sum_j \beta_j x_j, \qquad p = \frac{1}{1 + e^{-(\beta_0 + \sum_j \beta_j x_j)}}. \tag{18}$$

Equivale a una red sin capas ocultas con salida sigmoide. Cada coeficiente se interpreta así: al aumentar x_j en una unidad, las *odds* se multiplican por e^{β_j}, con las demás variables fijas.

Ejemplo: interpretar un coeficiente

En el proyecto, el coeficiente de `int_rate` es 1.773. Como la variable está escalada a $[0,1]$, pasar de la tasa mínima a la máxima multiplica las *odds* de impago por $e^{1.773} = 5.9$.

9.2 Árbol de decisión

Divide los datos con preguntas del tipo «¿`int_rate` $>$ 0.4?» y elige en cada nodo la división que más reduce la **impureza de Gini**:

$$G = 1 - \sum_c p_c^2. \tag{19}$$

Ejemplo: reducción de impureza

Un nodo con 100 préstamos y 15 impagos tiene $G = 1 - (0.85^2 + 0.15^2) = 0.255$. Una división lo separa en 60 préstamos con 3 impagos ($G = 1 - (0.95^2 + 0.05^2) = 0.095$) y 40 con 12 impagos ($G = 1 - (0.7^2 + 0.3^2) = 0.42$). La impureza ponderada queda en $0.6(0.095) + 0.4(0.42) = 0.225$, una reducción de 0.030. La **importancia** de una variable es la suma de estas reducciones en todos los nodos donde se usa.

9.3 *Random Forest*

Promedia muchos árboles, cada uno entrenado con una muestra *bootstrap* de los datos y considerando en cada división solo un subconjunto aleatorio de variables. Promediar árboles poco correlacionados reduce la varianza sin aumentar mucho el sesgo.

En nuestro proyecto

Parámetros usados: árbol con profundidad máxima 5; *Random Forest* con 150 árboles, 5 variables por división y 12 hojas como máximo. Ambos con y sin `class_weight`. En prueba, la logística, el *Random Forest* y las redes empataron (AUC de 0.675 a 0.677).

10 Interpretabilidad

10.1 Intrínseca frente a *post hoc*

Un modelo es **intrínsecamente interpretable** si su estructura se puede leer directamente, como los coeficientes de una logística o las reglas de un árbol pequeño. Los métodos ***post hoc*** explican un modelo ya entrenado desde afuera, sin mirar su interior. Así se explica una red neuronal.

10.2 SHAP

SHAP reparte la salida de un préstamo entre sus variables usando valores de Shapley de la teoría de juegos (Lundberg & Lee, 2017):

$$f(x) = \mathbb{E}[f(X)] + \sum_{j=1}^{M} \phi_j(x). \tag{20}$$

Aquí $\mathbb{E}[f(X)]$ es la salida promedio sobre un conjunto de referencia (*background*) y ϕ_j es lo que la variable j sube o baja la salida en ese préstamo. Esta igualdad se llama **aditividad**. Para una red de clasificación se trabaja en *log-odds*, donde la descomposición es aditiva, y se vuelve a probabilidad con $p = 1/(1+e^{-f})$.

Ejemplo: del SHAP a la probabilidad

En el préstamo más riesgoso de la prueba, $\mathbb{E}[f(X)] = -1.851$, es decir, $p = 1/(1+e^{1.851}) = 0.136$. Las contribuciones suman $+1.197$: `purpose_small_business` $+0.31$, `grade_F` $+0.23$, `term` $+0.18$, y otras. La salida queda en $f(x) = -0.654$, que corresponde a $p = 1/(1+e^{0.654}) = 0.342$. Un SHAP de $+0.31$ significa $+0.31$ en *log-odds*, no 31 puntos de probabilidad.

Importancia global SHAP: el promedio de $|\phi_j|$ sobre muchos préstamos. **Gráfico *beeswarm*:** un punto por préstamo; el color indica el valor de la variable y la posición, su efecto.

Idea clave

Coeficientes, importancias de impureza y SHAP describen cómo *el modelo* usa las variables, no relaciones causales. Si dos variables están correlacionadas (como la tasa y el grado), cada modelo puede repartir entre ellas el mismo efecto de forma distinta.

11 Tabla resumen de hiperparámetros

Hiperparámetro	Qué controla	Si es muy bajo / poco	Si es muy alto / mucho	Valor elegido
Optimizador	cómo se usa el gradiente	–	–	Adam
Tasa de aprendizaje	tamaño del paso	aprendizaje lento, se estanca	oscila o diverge	0.0001
Lote	gradiente por actualización	gradiente ruidoso, lento por época	menos actualizaciones, generaliza peor	32
Activación	no linealidad	–	–	tanh
Capas	profundidad	no capta relaciones complejas	desvanecimiento del gradiente, sobreajuste	20
Neuronas	ancho	poca capacidad	sobreajuste, costo	64
Dropout	fracción apagada	sin efecto regularizador	pierde capacidad, colapsa	0.25
L1 / L2 (λ)	tamaño de los pesos	sin efecto	pesos casi nulos, subajuste	L1, 10^{-4}
Paciencia	tolerancia antes de parar	se detiene por ruido	gasta tiempo, o sobreajusta sin restaurar	5
Épocas	pasadas por los datos	subajuste	sobreajuste si no hay parada	50
Pesos de clase	costo de cada error	ignora la clase rara con $t = 0.5$	sobredetecta (resuelto igual con el umbral)	sin pesos

Tabla 3. *Resumen de hiperparámetros, efectos típicos y valores elegidos en la red mejorada del proyecto.*

12 Preguntas de repaso

1. **Un modelo tiene 85 % de exactitud y F1 = 0. ¿Qué está pasando?**
   Predice siempre la clase mayoritaria. Con 85 % de préstamos pagados, decir siempre «pagado» da esa exactitud sin detectar ningún impago.

2. **¿Por qué AP y AUC no cambian al mover el umbral?**
   Porque dependen solo del orden de las probabilidades, y el umbral no cambia ese orden.

3. **¿Cuál es el AP de un modelo aleatorio en este problema?**
   La prevalencia de impago, 0.148. El AUC de un modelo aleatorio es 0.5.

4. **¿Por qué los pesos de clase no mejoraron el AP?**
   Porque desplazan todas las probabilidades por casi la misma cantidad en *log-odds*: cambian el F1 con $t = 0.5$, pero no el orden de los préstamos.

5. **¿Qué ventaja y qué desventaja tiene la búsqueda secuencial?**
   Ventaja: su costo es la suma, no el producto, de los valores probados. Desventaja: ignora interacciones y depende del orden de las etapas.

6. **¿Por qué conviene incluir el valor vigente en cada etapa?**
   Para que la etapa pueda conservarlo si ninguna alternativa mejora. En el proyecto, sin la opción de 0 en la etapa de *dropout*, el AP bajó de 0.2721 a 0.2520.

7. **¿Por qué una red de 100 capas puede quedar en AUC $\approx$ 0.5?**

Por el desvanecimiento del gradiente: el producto de muchas derivadas menores que 1 deja las primeras capas sin aprender.

8. **¿En qué se diferencian L1 y L2?**
   L2 achica los pesos en proporción a su tamaño. L1 empuja con fuerza constante y lleva a cero los pesos pequeños (selección de variables).

9. **¿Por qué todas las paciencias dieron el mismo resultado?**
   Porque todas restauraron los pesos de la misma mejor época; una paciencia mayor solo añadió épocas de espera.

10. **Si dos modelos muy distintos llegan al mismo AUC, ¿qué sugiere?**
    Que el límite está en la información de las variables y no en el modelo. Para mejorar, hay que construir mejores variables.

11. **¿Qué significa un SHAP de $+0.31$ en *log-odds*?**
    Que esa variable sube la salida en 0.31 *log-odds*. El cambio en probabilidad depende del punto de partida; no son 31 puntos porcentuales.

12. **Calcula el F1 con VP = 40, FP = 60 y FN = 60.**
    Precisión $= 40/100 = 0.4$; *recall* $= 40/100 = 0.4$; F1 $= 0.4$.

Referencias

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. En *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (pp. 249–256). PMLR.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR)*.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems 30* (pp. 4765–4774).

Prechelt, L. (1998). Early stopping—but when? En G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55–69). Springer.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE, 10*(3), e0118432.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*, 1929–1958.